

SOROTI UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

DESIGN, IMPLEMENTATION AND CALIBRATION OF A MULTI-SENSOR EDGE NODE

WSN2100 - WIRELESS SENSOR NETWORKS

Laboratory 1: Technical Report

Submitted by

DAVIS TWINOMUGISHA
SARAH ASINGWIIRE
OPUCHE SAMSON
PHILLIP OKWANY

in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING IN ELECTRONICS AND COMPUTER ENGINEERING

6 NOVEMBER 2026

TABLE OF CONTENTS

1 OBJECTIVE AND COMPETENCY....1
2 SYSTEM ARCHITECTURE AND HARDWARE....1
3 SENSOR INTERFACES AND SAMPLING DESIGN....2
4 SENSOR CHARACTERIZATION AND DATA QUALITY....3
5 EDGE PROCESSING AND LOCAL DECISION LOGIC....4
6 PYTHON DATA ANALYSIS AND VISUALIZATION....5
7 DISCUSSION AND ENGINEERING INTERPRETATION....7
8 LIMITATIONS AND RECOMMENDED IMPROVEMENTS....8
9 CONCLUSION....9
REFERENCES....9
APPENDIX: REPOSITORY LINK AND FILE STRUCTURE....10

1 OBJECTIVE AND COMPETENCY

The objective of this laboratory is to design, implement and validate a low-cost sensing node that acquires measurements from several sensors, checks their plausibility, processes data locally, makes a simple edge decision and streams machine-readable data for Python analysis. The node targets environmental monitoring in a smart building. The updated quantitative analysis uses the supplied 720-row Arduino recording, while the ESP32 Wokwi design remains available for simulation.

Competencies demonstrated: sensor interfacing on the ESP32 simulation and Arduino UNO R4 prototype; local filtering and threshold actuation; structured serial output; descriptive statistics and visualization in Python; and a reproducible repository. The supplied recording does not retain timestamps or original unfiltered temperature, so timing and raw-versus-filtered validation remain limited.

2 SYSTEM ARCHITECTURE AND HARDWARE

The system has sensing, processing, local indication, serial communication and Python analysis subsystems (Fig. 1). In the Wokwi design, a DHT22, photoresistor module and PIR sensor feed an ESP32 DevKit V1. Its firmware validates, calibrates, filters and classifies readings, then controls an LED and buzzer. Figure 2 retains the simulated circuit wiring. The physical prototype uses the equivalent processing workflow with the different sensors and pins described below.

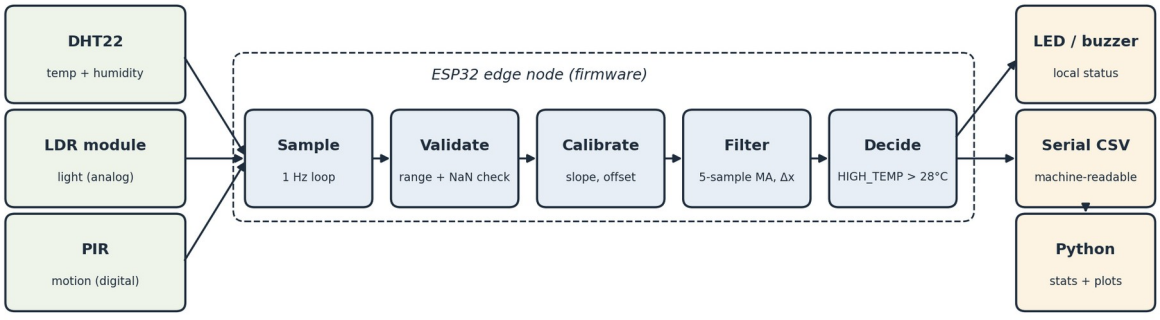


Fig. 1. ESP32 simulation architecture, from sensing to Python analysis.

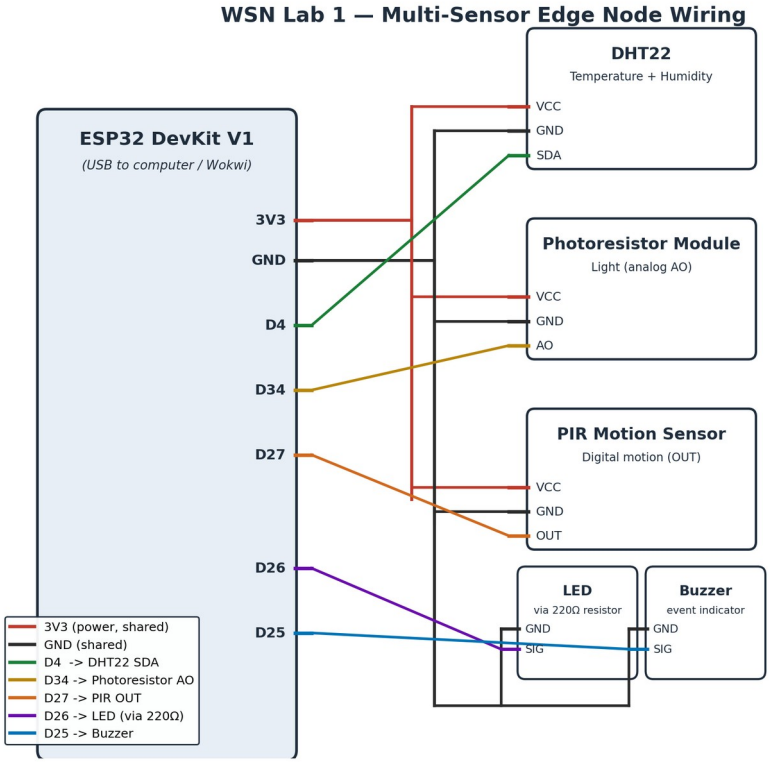


Fig. 2. Circuit/wiring diagram of the ESP32 simulated node.

Physical prototype: the Arduino UNO R4 WiFi [8] uses a KY-028 NTC thermistor module on A0, a two-pin LDR with a 10 kΩ divider on A1, PIR motion input on D7, red LED on D2 and active buzzer on D8. The supplied prototype firmware performs temperature conversion, offset correction, five-sample filtering and local actuation. Its humidity field is calculated from temperature because no humidity sensor is fitted. The updated figures and descriptive statistics use the supplied Arduino CSV rather than the archived Wokwi capture.

3 SENSOR INTERFACES AND SAMPLING DESIGN

Table 1 distinguishes the two implementations. The ESP32 schedules a nominal one-second loop using millis(). The DHT22 and its library provide fresh temperature and humidity readings at approximately two-second intervals [2], [4]. The UNO prototype uses delay(1000), with an additional 200 ms motion-alert delay in one branch; its sample interval therefore includes processing and alert time. The submitted CSV has no time or counter column, so neither the actual sampling frequency nor recording duration can be verified.

Device / channel	ESP32 pin	UNO R4 pin	Interface
Temperature	GPIO4, DHT22	A0, KY-028	Digital / analog
Humidity	GPIO4, DHT22	No RH sensor	Measured / estimated
Light	GPIO34 (ADC1)	A1	Analog
PIR motion	GPIO27	D7	Digital input
Red LED	GPIO26, 220 Ω	D2	Digital output
Buzzer	GPIO25	D8	Digital output

Table 1. Sensor interfaces and pin assignments in the two implementations.

The primary CSV columns are temperature, humidity, light_pct, motion, temp_status and motion_status. Reported temperature and humidity use degrees Celsius and percent RH respectively; light_pct is a relative percentage, not lux. Analysis adds sample_number and derived features without inventing timestamps. No second +0.95 °C correction is applied to the recorded temperature because its calibration state at capture is not documented.

4 SENSOR CHARACTERIZATION AND DATA QUALITY

The primary file contains 720 ordered records, exceeding the guide's 600-row minimum. Temperature ranges from 26.0 to 29.0 °C, reported humidity from 46.0 to 47.5%, and supplied light from 15.08 to 100.00%. The CSV does not identify controlled changes or a timed stable baseline. Table 2 describes the supplied channels. The separately generated light test values are excluded from this report's quantitative results.

Channel	Count	Mean	Min	Max	Std. dev.
Temperature (°C)	720	27.473	26.000	29.000	0.874
Reported humidity (%)	720	46.796	46.000	47.500	0.245
Light (%)	720	87.911	15.080	100.000	18.860

Table 2. Descriptive statistics of the supplied Arduino recording; sample standard deviation (ddof = 1).

Data quality: there are no missing source cells and no rows fail the implemented finite-number, range and binary-motion checks. One identical row is retained because repeated measurements are not evidence of duplicate logging. Without timestamps, dropped samples and timing irregularities cannot be detected. Zero flagged rows does not demonstrate calibrated accuracy or exclude faults hidden by firmware substitution of the last temperature.

Status consistency: 235 rows carry TEMP_HIGH, whereas 215 printed temperatures are strictly above 28.0 °C. The 20 disagreements all occur at printed 28.0 °C and may reflect rounding or unlogged device logic. Source labels remain unchanged and reconstructed flags are reported separately. Motion labels agree with the binary motion column. The firmware estimates humidity from temperature, so the reported humidity statistics do not establish independent RH sensing.

5 EDGE PROCESSING AND LOCAL DECISION LOGIC

The ESP32 firmware checks DHT22 validity, applies $T_{cal} = a \cdot T_{raw} + b$, calculates a five-sample moving average and a sample-to-sample difference, and makes a threshold decision. Its final temperature alarm uses 28.0 °C activation and 27.5 °C clearing thresholds. The UNO firmware has a separate implementation: it rejects specified near-rail ADC values internally, applies a +0.95 °C offset, averages five temperatures and selects TEMP_LOW below 18 °C, TEMP_HIGH above 28 °C, or TEMP_NORMAL otherwise. The UNO sketch does not implement the ESP32 hysteresis or export rate-of-change fields.

On the UNO prototype, high temperature turns on the red LED and continuous buzzer; motion otherwise causes a short buzzer pulse. An invalid thermistor conversion causes the filtering routine to return its previous temperature, but the current CSV does not expose a validity or stale-data flag. The Python analysis therefore reports its own numeric checks separately from device validity. No IMU is fitted, so IMU mean, RMS or standard deviation is not claimed.

Calibration of the prototype: the supplied pairs are 28.1 °C versus 29.3 °C and 32.3 °C versus 33.0 °C (Table 3). Their mean reference-minus-sensor difference is +0.95 °C. On these same two fitting pairs, the offset model reduces MAE from 0.950 to 0.250 °C and RMSE from 0.982 to 0.250 °C, reductions of 73.7% and 74.6%. These are fitting residuals, not independent validation accuracy. A two-point line fits both pairs exactly by construction. Figure 3 retains the offset model used by CAL_OFFSET = 0.95 in the prototype firmware.

Point	Prototype sensor (°C)	Thermometer (°C)	Reference - sensor (°C)
1	28.1	29.3	+1.2
2	32.3	33.0	+0.7

Table 3. Supplied prototype/reference calibration pairs.

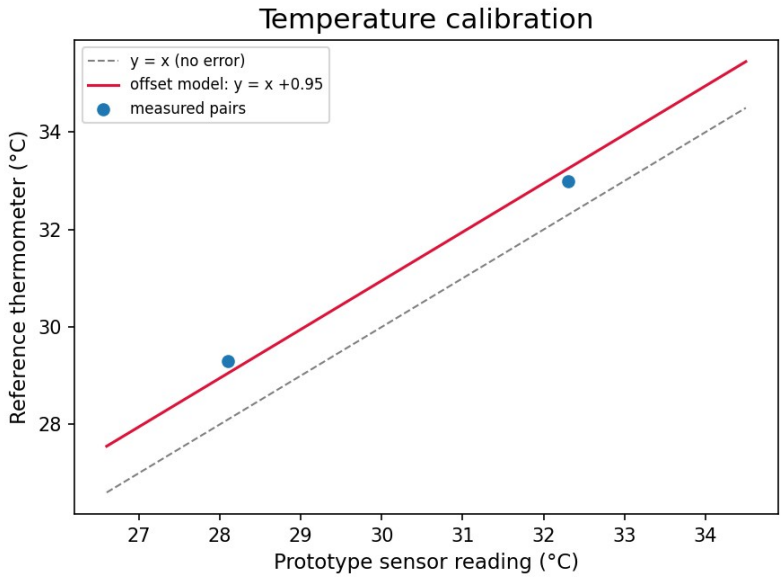


Fig. 3. Temperature calibration pairs and fitted offset model.

Offline reproduction: the recorded temperature receives an additional trailing five-sample mean. The first four means remain unavailable until five readings exist. Consecutive differences use ΔT = T[k] - T[k-1], in °C per sample. Per-second rates require timestamps or an explicit sample-interval assumption. Original unfiltered readings are absent, so this analysis cannot reconstruct them or prove new execution on the controller.

6 PYTHON DATA ANALYSIS AND VISUALIZATION

analysis.ipynb imports the primary Arduino CSV into a Pandas DataFrame [5], retains all rows and source columns, applies numeric checks, computes the required descriptive statistics and exports processed data. Matplotlib [6] generates Figs. 4–8. The archived Wokwi data and synthetic reference example remain separate. acquisition.ipynb documents the source import, and capture_serial.py supports future host-timestamped serial recordings. That capture utility was not used to create the supplied file.

Results: 720 observations; measured duration unavailable; no missing cells or rows flagged by the applied numeric checks. Source statuses contain 235 high-temperature samples across 160 observed runs, of which the first is already active and 159 rising edges occur inside the recording. Motion appears in 211 samples across 139 runs. There are 52 samples with both source conditions and 394 unique samples with either. Under the reconstructed strict >28 °C rule there are 215 positive samples across 155 runs; none exceed 30 °C.

Plot interpretation: the horizontal axes use sample number because timestamps were omitted. An assumed one-second interval would imply a 719-second first-to-last span, but this is not a

measured duration. The comparison plot shows recorded temperature versus additional filtering, rather than claiming that the source column contains raw sensor readings.

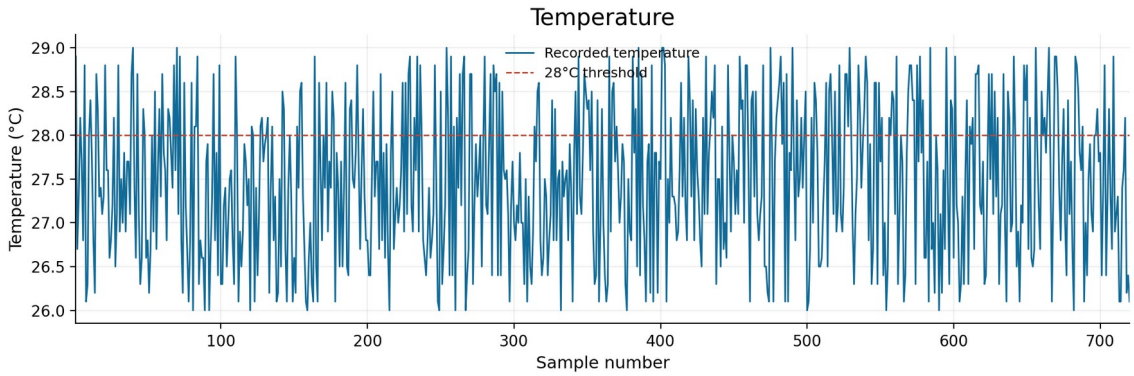


Fig. 4. Recorded temperature by sample number, with the 28 °C device threshold.

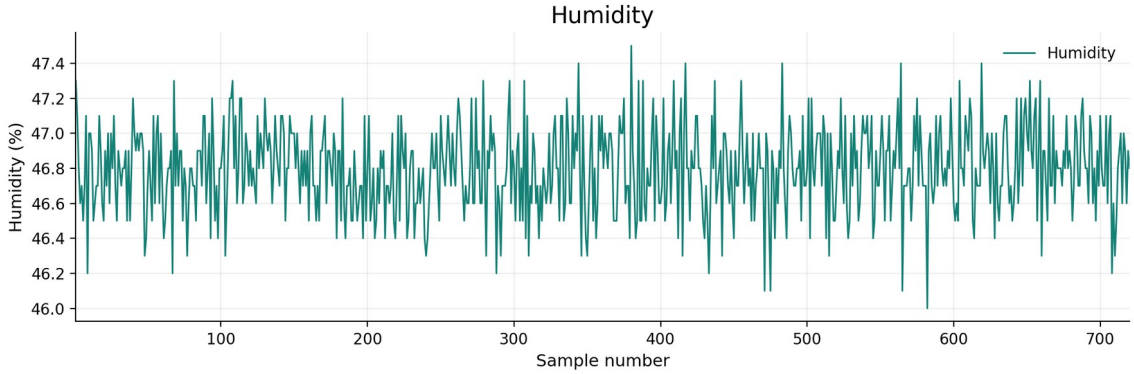


Fig. 5. Reported humidity by sample number; independent RH sensing is not established.

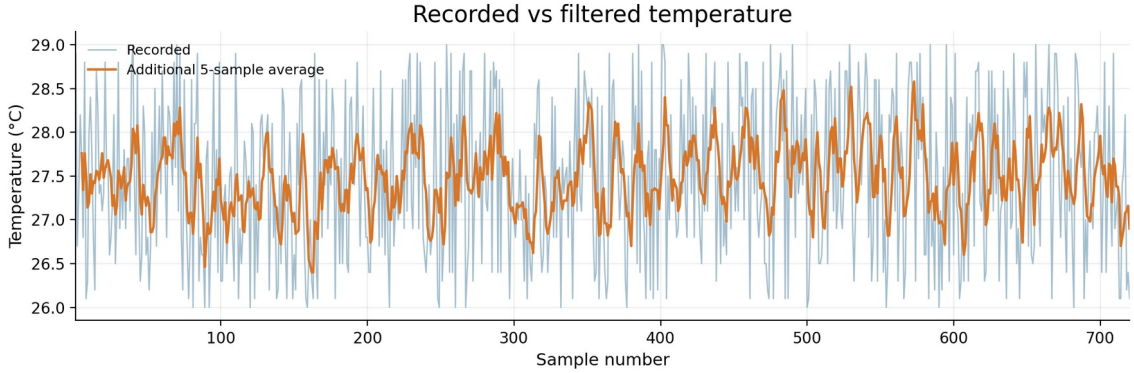


Fig. 6. Recorded temperature and an additional five-sample moving average.

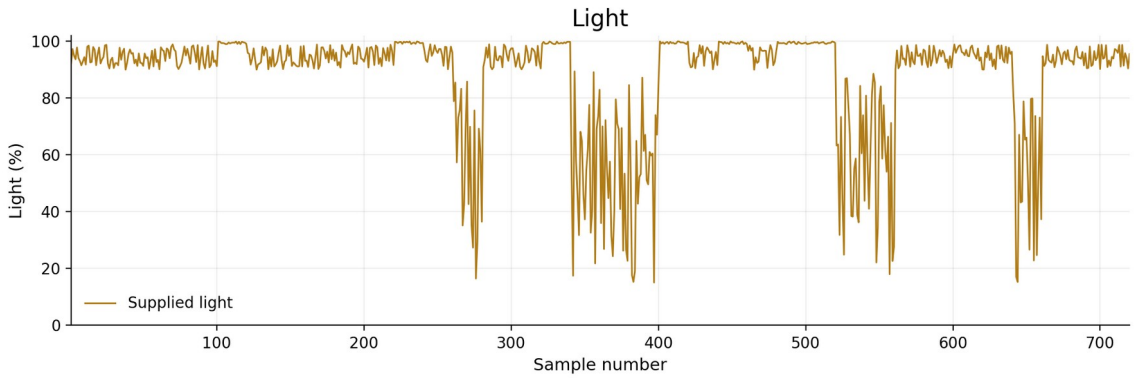


Fig. 7. Supplied light percentage by sample number; generated light values are excluded.

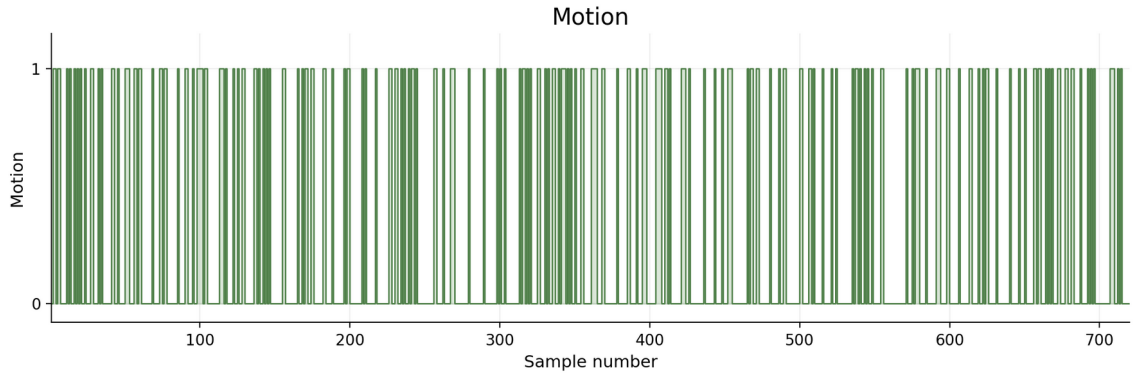


Fig. 8. Recorded PIR motion output by sample number.

7 DISCUSSION AND ENGINEERING INTERPRETATION

Q1. Major subsystems and their roles. Sensors provide environmental signals. The controller acquires and processes them, local indicators provide warnings, serial communication exports observations and Python computes statistics and plots. ESP32 simulation and UNO prototype wiring and firmware are documented separately.

Q2. Effective sampling rate. The supplied file cannot establish an effective rate because it has no timestamps. The ESP32 loop is nominally 1 Hz, with fresh DHT22 readings approximately every two seconds. The UNO loop includes blocking delays and processing, so its counter cannot be assumed to equal elapsed seconds.

Q3. Greatest variability. The supplied light channel spans 15.08–100.00%, with standard deviation 18.860 percentage points. Temperature spans 26.0–29.0 °C, with standard deviation 0.874 °C; reported humidity spans 46.0–47.5%, with standard deviation 0.245 percentage points. These describe within-channel spread. Standard deviations with different units are not a direct ranking of sensor noise.

Q4–Q5. Systematic error revealed by calibration; change in MAE and RMSE. The two supplied calibration pairs show the prototype reading 1.2 and 0.7 °C below the reference. Their

fitted +0.95 °C offset reduces same-pair MAE from 0.950 to 0.250 °C and RMSE from 0.982 to 0.250 °C. More points and held-out reference readings are needed to assess accuracy beyond those fitting pairs.

Q6. Effect of filtering. Over the 716 samples with complete averaging windows, recorded-temperature standard deviation is 0.874 °C, compared with 0.395 °C after additional averaging. This demonstrates smoothing of the supplied sequence, not an isolated measurement of noise rejection. A five-sample average retains earlier readings and introduces response lag. Seconds of delay cannot be measured without timestamps.

Q7. Edge vs. transmitted information. Immediate validity checks, conversion, calibration, filtering and alarm decisions should run locally. A deployed system can transmit periodic summaries and event records, while retaining selected raw and validity data for diagnosis. The current prototype exports serial rows and does not implement a wireless link.

Q8. Energy if every raw sample were transmitted. Frequent wireless transmissions can increase radio energy and network traffic. For illustration, 50 bytes per row at 1 Hz is 4.32 MB/day; one such row every five seconds is 0.864 MB/day before protocol overhead. These are assumed data-volume calculations, not measured packet sizes or battery-life results for this serial-only recording.

8 LIMITATIONS AND RECOMMENDED IMPROVEMENTS

Timing and baseline: the file exceeds 600 rows but contains no timestamps, so actual duration, missed observations, latency and a five-minute stable baseline are unverified. Record real device or host receipt times and document stable test intervals.

Humidity and light: the prototype code estimates humidity from temperature and reports relative light percentage. Add a humidity sensor and document the LDR conversion and polarity. Use a reference instrument if lux is required.

Raw channels and validity: the CSV omits original unfiltered temperature, thermistor ADC counts and hardware validity flags. Log these alongside filtered values and expose stale substitutions so raw-versus-filtered and fault behavior can be assessed.

Calibration range: only two pairs cover sensor readings of 28.1–32.3 °C. Their residual errors do not establish general accuracy. Add reference pairs over a wider range and evaluate on additional independent measurements.

Status boundaries: 20 TEMP_HIGH labels accompany printed 28.0 °C. Export greater precision and the exact decision variable to determine whether rounding or other firmware logic explains the discrepancy.

Hardware and scope: the physical build uses an UNO R4 WiFi, while the guide requests an ESP32-class prototype. The Wokwi design uses ESP32. IMU processing, buffering and MQTT remain outside the demonstrated work.

9 CONCLUSION

The submission now combines the original ESP32 simulation and UNO R4 prototype with analysis of the supplied 720-row Arduino CSV. The updated notebook preserves observations, computes descriptive statistics and event counts, and provides five plots. It reproduces additional filtering and sample-to-sample changes offline. The supplied two-point calibration supports a +0.95 °C offset and smaller fitting residuals. Remaining evidence concerns actual timing, original raw channels, independent humidity sensing, expanded calibration and direct verification of device validity and local processing.

REFERENCES

[1] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Aosong Electronics, “Digital-output relative humidity & temperature sensor/module DHT22 (AM2302).” [Online]. Available: https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf

[3] Wokwi, “Wokwi Docs: ESP32 simulation and parts reference.” [Online]. Available: https://docs.wokwi.com

[4] Adafruit Industries, “DHT sensor library,” GitHub repository. [Online]. Available: https://github.com/adafruit/DHT-sensor-library

[5] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf. (SciPy 2010), 2010, pp. 56–61.

[6] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[7] Soroti University, School of Engineering and Technology, “Wireless Sensor Networks, Laboratory Guide 1: Design, Implementation and Calibration of a Multi-Sensor Edge Node,” 2026.

[8] Arduino, “UNO R4 WiFi,” Arduino Documentation. [Online]. Available: https://docs.arduino.cc/hardware/uno-r4-wifi

APPENDIX: REPOSITORY LINK AND FILE STRUCTURE

Repository: https://github.com/davis-twinomugisha/wsn_lab1

Wokwi simulation: https://wokwi.com/projects/476121306266761217

Primary recording: data/raw/sensor_data_light_regimes_720rows.csv. All 720 rows and the original six columns are retained in the raw file.

```
wsn_lab1/
  firmware/       ESP32 simulation and UNO R4 prototype
  python/         acquisition, analysis and calibration
  data/raw/       Arduino recording and archived Wokwi data
  data/processed/ cleaned_log.csv, statistics and summaries
  data/calibration/ calibration_pairs.csv
  figures/        five updated plots and calibration fit
  docs/           report PDF, DOCX, builder and diagrams
  README.md       setup and reproduction instructions
```